

TESI FINALE DI PROGETTAZIONE DI SISTEMI OPERATIVI

Programmazione concorrente nel Linguaggio Java

Approfondimento sull'utilizzo dei *Thread* in Java

Studente:

Tommaso Cosimi

Matricola 191739

Docente:

Prof.ssa Letizia Leonardi

Codocente:

PhD Silvia Cascianelli

Sommario

Nel corso della Storia dell'Informatica, l'avanzamento tecnico e tecnologico verificatosi nell'ambito delle Architetture dei Calcolatori e dei Sistemi Operativi ha permesso grandi passi in avanti nelle dinamiche di utilizzo degli Elaboratori Elettronici. Si è assistito al passaggio dall'esecuzione sequenziale delle operazioni ad un'esecuzione parallela delle stesse, il tutto all'interno di un singolo Elaboratore.

L'opportunità di avere esperienze utente multiprogrammate è stata resa possibile, oltre che dagli Elaboratori non sequenziali, anche da Linguaggi di Programmazione anch'essi non sequenziali che permettano l'esecuzione concorrente di più processi. Tale flessibilità ha permesso un grado di utilizzo delle risorse nettamente superiore al passato, permettendo prestazioni superiori a parità di risorse.

Lo scopo di questo elaborato sarà quello di approfondire l'argomento della Programmazione Concorrente nel Linguaggio Java, riportando allo stesso tempo esempi pratici di utilizzo.

Indice

1	Introduzione	11
1.1	Algoritmo, Programma, Processo	11
1.2	Esecuzione Sequenziale e non Sequenziale	13
1.2.1	Processi Sequenziali	14
1.2.2	Processi non Sequenziali	14
1.3	Processi Interagenti	15
1.3.1	Interazione Indiretta (Competizione)	15
1.3.2	Interazione Diretta (Cooperazione)	16
1.3.3	Interazione Indesiderata (Interferenza)	16
1.4	Sincronizzazione tra Processi	17
1.4.1	Sezione Critica	17
1.4.2	Strumenti di Sincronizzazione	17
1.4.3	Modelli Logici per la Concorrenza	23
1.5	Processi Pesanti e Processi Leggeri	23
1.5.1	Processi Pesanti	24
1.5.2	Processi Leggeri	24

Elenco delle figure

1.1	Grafo delle Precedenze di un Processo Sequenziale.	14
1.2	Grafo delle Precedenze di un Processo non Sequenziale. . . .	15
1.3	Esempio di Interazione Competitiva tra due Processi P1 e P2.	16

Elenco delle tabelle

Elenco degli algoritmi

1	<i>Bubble Sort</i>	12
2	Esempio di utilizzo di un Semaforo	18
3	Pseudocodice di un Semaforo	19
4	Pseudocodice di un Monitor	20

Capitolo 1

Introduzione

Il seguente Capitolo fornirà un'introduzione teorica al concetto di esecuzione non sequenziale di un processo, ponendo enfasi sui vantaggi portati e sulle nuove necessità richieste da tale implementazione.

1.1 Algoritmo, Programma, Processo

Nella definizione della singola unità di esecuzione si ritiene opportuno effettuare una leggera digressione.

Algoritmo

Un Algoritmo è un procedimento puramente logico studiato appositamente per la risoluzione di uno specifico problema. In letteratura sono rintracciabili moltissimi Algoritmi, per brevità vengono nominati a titolo di esempio Algoritmi di Ordinamento come il *Bubble Sort*[1] - di cui viene fornito lo pseudocodice a seguire - od il *Radix Sort*[2], e Algoritmi di Ricerca del Cammino Minimo in un Grafo come l'*Algoritmo di Dijkstra*[3] o l'*Algoritmo di Bellman-Ford*[4].

Algoritmo 1: Ordinamento per confronti “*Bubble Sort*”

```

Data: A;           // Vettore di numeri interi non ordinati
Result: A;         // Vettore di numeri interi ordinati
1 partizioneNonOrdinata  $\leftarrow$  A.length;
2 while partizioneNonOrdinata > 1 do
3   for i = 2 fino a k do
4     if A[i] < A[i - 1] then
5       |   Scambia A[i] ed A[i - 1];
6     end if
7   end for
8   partizioneNonOrdinata  $\leftarrow$  (ultimoScambio - 1);
9 end while
10 return A

```

Programma

Un Programma è la descrizione in codice di un Algoritmo in uno specifico Linguaggio di Programmazione in maniera tale da permetterne l'esecuzione all'interno di un Calcolatore, è un'entità statica. Si riporta di seguito un'esempio di Programma che descriva l'Algoritmo di Ordinamento per confronti “*Bubble Sort*” implementato tramite il Linguaggio di Programmazione Java.

```

1 public class BubbleSort {
2
3     public static void BubbleSort(int[] A) {
4
5         // Definizione della dimensione della partizione del
6         //  $\hookrightarrow$  vettore non ordinata
7         int unorderedPartition = A.length;
8
9         // Ordinamento
10        while(unorderedPartition > 1) {
11            // Scorro la parte non ordinata
12            for(int i=2; i<unorderedPartition; i++) {
13                // Se non e' ordinato, scambia
14                if(A[i] < A[i-1]) {
15                    int buf = A[i];

```

```
15         A[i] = A[i-1];
16         A[i-1] = buf;
17     }
18 }
19 // Dopo lo scambio, riduco la dimensione della
    ↪ partizione del vettore non ordinata
20 unorderedPartition = unorderedPartition - 1;
21 }
22
23 }
24
25 }
```

Processo

Un Processo è la sequenza di operazioni eseguite da un Elaboratore che opera sotto il controllo di un Programma, è un'entità dinamica: in maniera intuitiva è possibile affermare che in linea generale un Processo è un Programma in esecuzione. Di seguito si riporta un esempio di esecuzione sequenziale di due Processi che eseguono lo stesso Programma, ovvero *Bubble Sort*.

```
[tcosimi@VivoBook-Arch]:~$ java BubbleSort 1 2 3 6 5 2
Il vettore non ordinato e': [1, 2, 3, 6, 5, 2]
Il vettore ordinato e': [1, 2, 2, 3, 5, 6]
[tcosimi@VivoBook-Arch]:~$ java BubbleSort 1 9 8 3 6 7
Il vettore non ordinato e': [1, 9, 8, 3, 6, 7]
Il vettore ordinato e': [1, 3, 6, 7, 8, 9]
```

1.2 Esecuzione Sequenziale ed Esecuzione non Sequenziale

La sequenza di operazioni eseguite da un processo può essere rappresentata attraverso un Grafo Orientato che indichi la precedenza delle une sulle altre. In questa visualizzazione i nodi indentificano il singolo evento da verificarsi, mentre gli archi l'ordinamento temporale tra i nodi.

1.2.1 Processi Sequenziali

Facendo riferimento alla rappresentazione descritta in precedenza, un Processo Sequenziale è tale da poter essere schematizzato tramite l'ausilio di un Grafo ad Ordinamento Totale: una particolare tipologia di Grafo ove ogni nodo - e quindi ogni evento - fatta eccezione per il nodo di inizio ed il nodo di fine, possiede un solo predecessore ed un solo successore.

A seguire viene effettuato un esempio di Processo Sequenziale.

Esempio di Rappresentazione di un Processo Sequenziale

Si riporta a titolo di esempio un Processo che richiede all'utente due numeri interi e ne restituisce la somma. Esso è rappresentabile come una sequenza ordinata di operazioni atomiche come esplicitato a seguire.

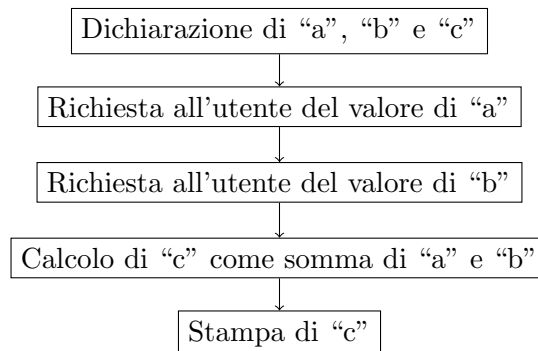


Figura 1.1: Grafo delle Precedenze di un Processo Sequenziale.

1.2.2 Processi non Sequenziali

Al contrario della controparte, un Processo non Sequenziale può essere rappresentato tramite l'ausilio di un Grafo ad Ordinamento Parziale: ogni nodo interno potrà avere più predecessori e più successori. Questo è possibile in quanto alcune operazioni eseguite dal Processo sono indipendenti tra loro o non è interessante il loro ordine di terminazione per l'utente finale.

A seguire viene effettuato un esempio di Processo non Sequenziale.

Esempio di Rappresentazione di un Processo non Sequenziale

Si supponga di dover valutare l'espressione matematica:

$$(5 + 8) + (6 - 3) * (9 + 2)$$

Ovviamente è possibile eseguire sequenzialmente la valutazione verificando prima i risultati delle operazioni tra parentesi, poi la moltiplicazione tra gli ultimi due termini ed infine l'addizione tra i primi due. La natura del problema permette tuttavia di eseguire in ordine arbitrario dei sottoinsiemi di operazioni, i quali risultati risultano indipendenti da un punto di vista di utilizzo dei dati: è possibile in effetti calcolare in ordine arbitrario o addirittura delegare la valutazione a differenti agenti di calcolo le espressioni elementari all'interno delle parentesi.

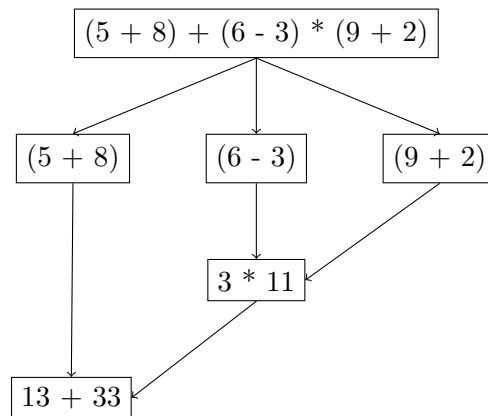


Figura 1.2: Grafo delle Precedenze di un Processo non Sequenziale.

1.3 Processi Interagenti

Dall'esecuzione non sequenziale è possibile ottenere situazioni ove due o più Processi devono interagire tra loro per il completamento delle rispettive operazioni. È possibile individuare tre possibili tipi di interazione.

1.3.1 Interazione Indiretta (Competizione)

In questo caso i suddetti Processi competono per l'utilizzo di risorse comuni come variabili o dispositivi di Input/Output. Un esempio lampante potrebbe

essere la condivisione di una variabile numerica per due Processi P1 e P2.

La modifica contemporanea della Variabile Globale “x” da parte di P1 e

```
// Variabile Globale  
int x = 10;
```

Esecuzione di P1:

```
...  
x = x + 5;  
...
```

Esecuzione di P2:

```
...  
x = x + 10;  
...
```

Figura 1.3: Esempio di Interazione Competitiva tra due Processi P1 e P2. Entrambi i Processi illustrati vanno ad operare sulla Variabile Globale “x”.

P2 potrebbe risultare problematica in quanto la lascerebbero in uno stato inconsistente.

Alla base di questa interazione si trova dunque il problema della Mutua Esclusione: una risorsa può essere utilizzata da uno e un solo Processo alla volta. Al fine di una corretta esecuzione bisognerà impossibilitare l'esecuzione contemporanea di Operazioni che facciano utilizzo di risorse comuni.

1.3.2 Interazione Diretta (Cooperazione)

La Cooperazione prevede invece lo scambio delle informazioni da un Processo ad un altro. Si prenda in esame la Figura 1.2: il Processo che andrà a valutare l'espressione finale dovrà sicuramente ottenere i risultati del Processo che ha calcolato la prima parentesi e di quello che ha calcolato la moltiplicazione immediatamente successiva.

Al fine di preservare la correttezza dell'Operazione si renderà necessario far iniziare le Operazioni del Processo finale successivamente al termine delle Operazioni degli altri due Processi.

1.3.3 Interazione Indesiderata (Interferenza)

Quest'ultima è un effetto indesiderato dato da errori di sincronizzazione tra i processi.

L'Interferenza viene risolta tramite l'utilizzo dei costrutti di sincronizzazione atti alla risoluzione dei problemi nati dalla Competizione e dalla Cooperazione citati in precedenza.

1.4 Sincronizzazione tra Processi

Alla luce di quanto esposto risulta importante esplicitare quali siano gli strumenti e le tecniche di sincronizzazione utilizzate per garantire la correttezza dei risultati di Processi Concorrenti. Prima di passare ad una loro descrizione è comunque necessario introdurre il concetto di Sezione Critica.

1.4.1 Sezione Critica

La Sezione Critica viene definita come una serie di Operazioni nelle quali un Processo accede e/o modifica risorse comuni a più Processi.

Diverse Sezioni Critiche che accedano a gruppi differenti di Risorse possono essere scisse in altrettante categorie.

1.4.2 Strumenti di Sincronizzazione

Gli Strumenti illustrati di seguito non rappresentano l'interezza delle metodologie di Sincronizzazione disponibili in letteratura, ma sono stati scelti in maniera tale da essere il più possibile utili al fine di introdurre i concetti alla base del *Multi-Threading* nel Linguaggio Java ed inoltre chiarificare alcune metodologie utilizzate anche al di fuori di quest'ultimo (si veda lo Scambio di Messaggi).

Lo scopo di tali Meccanismi sarà quello di permettere al più ad un processo di accedere ad ogni categoria di Sezione Critica. Per ottenere questo risultato, la Sezione Critica va incapsulata da un Prologo che la preceda ed un Epilogo che la segua. In maniera intuitiva si può pensare al Prologo e all'Epilogo come l'operazione di acquisizione di un lucchetto per bloccare una risorsa per poi rilasciarlo al prossimo Processo che ne necessita rispettivamente.

A seconda del tipo di Interazione tra i Processi presi in esame è importante definire la corretta metodologia di Sincronizzazione: per Interazioni di tipo Competitivo si utilizzano tipicamente Strumenti quali Semafori e Monitor, mentre per Interazioni di tipo Cooperativo si usufruisce della possibilità di

scambio di Messaggi tra Processi. Si riportano a seguire i tre Meccanismi citati.

Semaforo

Il Semaforo[5], introdotto dall'informatico olandese Edsger Wybe Dijkstra, è un tipo di dato astratto (ADT) atto alla sincronizzazione che si compone di:

- Valore Intero $S_v \geq 0$;
- Coda di Descrittori Q_s .

E sul quale è possibile effettuare le operazioni di:

- “wait(S)”;
- “signal(S)”.

Il Semaforo va ad incapsulare la Sezione Critica al fine di rendere possibile ad un solo processo alla volta l'accesso ad essa.

La variabile S_v mantiene come informazione quanti processi possono accedere alla loro Sezione Critica (se vale zero il processo si sospende sul Semaforo), mentre Q_s è una Coda di Descrittori dei Processi che non sono riusciti ad accedere alla loro Sezione Critica a causa del Semaforo.

Le Operazioni “wait(S)” e “signal(S)” vanno invece a decrementare ed aumentare il valore della variabile S_v del Semaforo.

L'utilizzo di un Semaforo per la protezione di una Sezione Critica avviene secondo il seguente schema:

Algoritmo 2: Esempio di utilizzo di un Semaforo

```

Data: S;                                     // Semaforo
1 wait(S);
2 Esecuzione della Sezione Critica;
3 signal(S);

```

Viene riportato a titolo di esempio un possibile pseudocodice per l'implementazione di un Semaforo.

Algoritmo 3: Pseudocodice di un Semaforo

```

1 Function Costruttore(int valoreIniziale):
2   this.Qs = Nuova Coda di Descrittori di Processo vuota;
   // Si richiede un valore iniziale da assegnare al
   Semaforo
3   this.Sv = valoreIniziale;
4 Function wait(Semaphore S):
   // Verifica che il valore di Sv sia > 0 e decrementalo,
   altrimenti inserisci il processo nella coda Qs
5   if S.Sv > 0 then
6     | S.Sv ← (S.Sv - 1);
7   else
8     | Imposta lo stato del Processo su “Blocked” ed inserisci il suo
       | descrittore in S.Qs;
9   end if
10 Function signal(Semaphore S):
   // Verifica che la coda Qs non sia vuota e libera
   l’ultimo processo per poterne riprendere
   l’esecuzione, altrimenti incrementa il valore di Sv
11 if ∃ Processo nella Coda S.Qs then
12   | Rimuovi il descrittore del Processo da S.Qs ed imposta il suo
       | stato su “Ready”;
13 else
14   | S.Sv ← (S.Sv + 1);
15 end if

```

Monitor

Il Monitor[6] è anch’esso un ADT che tiene conto dei Vincoli di Sincronizzazione: implementa infatti da sé meccanismi di Mutua Esclusione per cui al suo interno potrà esserci al più uno e un solo Processo attivo. Una peculiarità nell’utilizzo del Monitor rispetto a quello di un più semplice Semaforo è quella di poter nascondere lo stato interno delle risorse condivise tra i vari processi, lasciando loro visibili unicamente le operazioni effettuabili su di esse.

In maniera più precisa un Monitor “è un costrutto che associa un insieme di

procedure ad una struttura dati comune a più Processi” [7]. Esso è composto dai seguenti elementi:

- Costruttore, che genera un’entità di tipo Monitor;
- Variabili Locali, le quali vanno a definire lo stato di un’istanza del tipo Monitor;
- Procedure, per l’interfacciamento con le risorse condivise dai Processi.

Un possibile pseudocodice per un Monitor potrebbe essere simile a quanto segue:

Algoritmo 4: Pseudocodice di un Monitor

```
// Variabili di Condizione
1 Condition variabileCondizione1;
2 Condition variabileCondizione2;
3 ...;
4 Condition variabileCondizioneN;
// Costruttore
5 Function Costruttore():
    | // Codice del Costruttore
// Procedure
6 Procedure Procedura1(Parametro1, Parametro2, ..., ParametroJ):
    | // Codice della Procedura 1
7 Procedure Procedura2(Parametro1, Parametro2, ..., ParametroK):
    | // Codice della Procedura 2
8 ...;
9 Procedure ProceduraN(Parametro1, Parametro2, ..., ParametroL):
    | // Codice della Procedura N
```

Un Monitor dispone di due livelli di Controllo per l’esecuzione esclusiva di operazioni sulle Risorse condivise:

1. Primo Livello di Controllo:

Le procedure di un Monitor sono eseguite in maniera Mutualmente Esclusiva, per cui qualora nel Monitor fosse presente un Processo in esecuzione ed un qualsiasi altro processo esterno volesse accedervi, quest’ultimo verrebbe posto in stato “*Blocked*” e si “addormenterebbe” all’esterno di esso.

2. Secondo Livello di Controllo:

È possibile per un Processo all'interno di un Monitor di "addormentarsi" su una Variabile di Condizione in quanto potrebbe non rispettare i requisiti imposti. In questo caso quest'ultimo passerà allo stato "*Blocked*" e posto in coda ad essa, così da lasciare spazio ad un altro Processo dall'esterno per poter eseguire.

Le variabili di Condizione di un Monitor dispongono di due primitive "*Wait*" e "*Signal*" che, seppur molto simili a quelle di un Semaforo, godono di leggere differenze: in questo caso infatti la "*Wait*" ha sempre un significato bloccante, mentre la "*Signal*" non produce effetti qualora non ci siano processi in coda.

Scambio di Messaggi

Altro meccanismo di Sincronizzazione tra Processi Interagenti è lo scambio di Messaggi, utile in caso di interazioni di tipo Cooperativo. Per descriverne il funzionamento è importante definire alcune componenti quali le Primitive di Comunicazione "*Send*" e "*Receive*" ed il Canale di Comunicazione.

Vista la natura autoesplicativa delle Primitive, si pone ora enfasi sulla definizione del Canale di Comunicazione. Esso deve andare a determinare quali siano le coppie di Processi comunicanti e come identificarle (e quindi scegliere se utilizzare la Struttura Mailbox), se fruire o meno della "*Bufferizzazione*" e la lunghezza dei messaggi. Si noti che ciascuna di queste decisioni debba - almeno in linea teorica - essere ortogonale alle altre due.

Le scelte implementative sopracitate vengono analizzate a seguire:

- **Tecnica di Identificazione dei Processi Comunicanti**

L'Identificazione dei Processi "*Sender*" e "*Receiver*" può avvenire direttamente che non.

Nel primo caso si va ad utilizzare l'identificativo dei processi come parametro delle primitive, come sotto riportato.

```
/*
* R ed S sono rispettivamente gli Identificatori del
  ↪ Processo "Sender" e del Processo "Receiver".
* La Variabile "message" sta ad indicare il messaggio
```

```

    ↪ inviato dal Processo "Sender" e che il Processo "
    ↪ Receiver" dovrà ricevere.
*/
send(R, message);
receive(S, message);

```

È anche possibile creare uno schema di tipo “Molti a Uno”, “Uno a Molti” o “Molti a Molti” tramite l'utilizzo di speciali Identificatori all'interno delle primitive “send” e “receive”.

```

// Molti a Uno
send(R, message); // Invia ad uno specifico Ricevitore
receive(ID, message); // Riceve da qualsiasi Mittente
// Uno a Molti
send(setOfReceivers, message); // Invia a piu' Ricevitori
receive(S, message); // Riceve dallo specifico Mittente "
    ↪ S"
// Molti a Molti
send(setOfReceivers, message); // Invia a piu' Ricevitori
receive(ID, message); // Riceve da qualsiasi Mittente

```

Per implementare un tipo di Identificazione Indiretta si va ad interporre un interlocutore terzo - la “Mailbox” - che si occupa di permettere ai Mittenti di inviare lei un messaggio, il quale verrà letto dai Destinatari a lei collegati.

- **Utilizzo della tecnica di “Bufferizzazione”**

L'utilizzo di tale funzionalità permette una comunicazione asincrona che consente un maggiore parallelismo all'applicativo. Il “Buffer” può essere di dimensione limitata o infinita. È possibile l'utilizzo di Messaggi di “Acknowledgement” - o ACK - da parte del Processo “Receiver” per confermare la ricezione del Messaggio. La corretta sequenza di ricezione potrebbe non essere garantita in caso di *Routing* Dinamico dei Messaggi, per cui si rende necessario l'utilizzo di una numerazione per l'ordinamento.

- **Scelta della lunghezza dei Messaggi**

È infine possibile scegliere se implementare dei Messaggi a lunghezza

fissa, rendendo l'implementazione semplice ma sprecando memoria; o utilizzare Messaggi di lunghezza variabile, che risolvono il problema dei primi al costo di una più complessa realizzazione.

1.4.3 Modelli Logici per la Concorrenza

Nel tempo sono stati definiti due Modelli Logici di riferimento per la gestione concorrente dei Processi: il Modello ad Ambiente Globale ed il Modello ad Ambiente Locale.

Modello ad Ambiente Globale

In tale contesto un'applicazione viene descritta come un insieme di Processi e Risorse, ove queste ultime sono una qualsiasi entità fisica o logica di cui uno o più Processi dell'applicazione stessa necessitano per portare a termine la loro esecuzione.

Data la natura condivisa di questo Ambiente sono possibili interazioni sia competitive che cooperative per l'utilizzo e lo scambio di risorse tra i Processi.

Modello ad Ambiente Locale

Definito successivamente, in questo Modello un'applicazione viene descritta come un insieme di Processi, ognuno dei quali va ad operare nel proprio ambiente utilizzando le proprie risorse, inaccessibili agli altri Processi della stessa applicazione.

La diversa concezione porta alla semplificazione delle interazioni tra i processi, rendendo possibili unicamente quelle di tipo cooperativo.

1.5 Processi Pesanti e Processi Leggeri

All'interno di un Sistema Operativo è possibile individuare due tipologie di Processi in esecuzione, ognuna di esse con caratteristiche distintive:

- Processi Pesanti;
- Processi Leggeri.

1.5.1 Processi Pesanti

I Processi Pesanti sono identificati tramite la ennupla:

$$\{Program\ Counter, Registri, Stack, Codice, Dati\}$$

Processi Pesanti distinti eseguono codice distinto ed accedono a dati distinti, non condividendo memoria ed ispirandosi quindi al Modello ad Ambiente Locale.

1.5.2 Processi Leggeri

I Processi Leggeri - o *Thread* - sono invece identificati dalla ennupla:

$$\{Program\ Counter, Registri, Stack\}$$

Più *Thread* correlati da un punto di vista logico condividono un'area di dati e codice tra loro formando un *Task* e ciascuno di essi rappresenta differenti contesti di esecuzione che risultano indipendenti all'interno di uno stesso *Task*. Da ciò segue che i *Task* sono invece identificati da una ennupla del tipo:

$$\{Thread_1, Thread_2, \dots, Thread_n, Codice, Dati\}$$

Alla luce di quanto illustrato è possibile immaginare un Processo Pesante come un *Task* composto da un solo *Thread*.

Si noti come sia comunque necessario, ai fini del *Context Switching*, mantenere un Descrittore di Processo (*Process Control Block* - *PCB*) per ogni Processo Leggero, rendendo più complesso il *PCB* del *Task* che va ad incorporare anche quelli dei suoi relativi *Thread*.

In termini implementativi, i *Thread* di uno stesso *Task* condividono lo spazio di indirizzamento, le variabili globali e la tabella dei File aperti proprie del *Task*, ispirandosi quindi al Modello ad Ambiente Globale. Questa somiglianza con tale ambiente implica l'insorgenza di problematiche quali le cosiddette "*Race Conditions*" date da interazioni di tipo sia Competitivo che Cooperativo: analogamente a quanto anticipato in precedenza, andranno di conseguenza implementate misure atte alla sincronizzazione tra i vari Processi Leggeri del *Task*.

Bibliografia

- [1] Wikipedia. Bubble Sort, Giugno 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Bubble_sort. Pagina visitata il 11 novembre 2023.
- [2] Wikipedia. Radix Sort, Maggio 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Radix_sort. Pagina visitata il 11 novembre 2023.
- [3] Wikipedia. Algoritmo di Dijkstra, Aprile 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Algoritmo_di_Dijkstra. Pagina visitata il 11 novembre 2023.
- [4] Wikipedia. Algoritmo di Bellman-Ford, Maggio 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Algoritmo_di_Bellman-Ford. Pagina visitata il 11 novembre 2023.
- [5] Wikipedia. Semaforo, Agosto 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Semaforo_\(informatica\)](https://it.wikipedia.org/wiki/Semaforo_(informatica)). Pagina visitata il 11 novembre 2023.
- [6] Wikipedia. Monitor, Marzo 2022. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Monitor_\(sincronizzazione\)](https://it.wikipedia.org/wiki/Monitor_(sincronizzazione)). Pagina visitata il 11 novembre 2023.
- [7] Prof. Letizia Leonardi. Costrutti di Sincronizzazione, Ottobre 2022. Dispensa disponibile all'url: <https://www.didattica.agentgroup.unimo.it/didattica/ProgettazioneS0/Lucidi/CostruttiDISincronizzazione-3bis.pdf>. Documento Protetto da Password. Pagina visitata il 11 novembre 2023.